SmartLift: A Reinforcement Learning-Based Elevator Control System with MPC Benchmarking

Abdelrahman Zidan, Mohamed Ahmed Gamal, Mohamed Ahmed El Sawy, Marwan Mohamed, Karim Elesseily

Department of Mechatronics Engineering, German University in Cairo, Egypt

Abstract

Abstract*—*This paper presents SmartLift, a reinforcement learning-based elevator control system formulated as a Markov Decision Process (MDP). A custom Python simulation environment models a 10-floor building with stochastic Poisson passenger arrivals ($\lambda = 0.3$). Two tabular RL algorithms—Q-Learning and SARSA—are trained for 5,000 episodes and evaluated against three baseline controllers: First-Come-First-Served (FCFS), Nearest-Request, and Model Predictive Control (MPC). A compact binary state representation bounds the practical state space to approximately 500 states, enabling full convergence within the training budget. SARSA achieves the best average waiting time (5.36 steps) and passenger throughput (148.3 served per episode), outperforming FCFS (8.47 steps, 137.6 served). MPC achieves competitive waiting time (5.67 steps) using 25% fewer movements than RL agents, demonstrating complementary strengths of learning-based and planning-based control. Critical engineering challenges—including state space explosion, reward shaping failures, and subtle Python bugs—are documented as a core contribution.

Index Terms*—*Reinforcement Learning, Elevator Control, Markov Decision Process, Q-Learning, SARSA, Model Predictive Control, Tabular RL.

I. Introduction

Elevator systems are a fundamental component of modern buildings. As occupancy becomes denser, efficient elevator traffic management grows increasingly important. Traditional strategies such as First-Come-First-Served (FCFS) or nearest-floor heuristics are straightforward but fail to adapt to stochastic demand patterns.

Reinforcement Learning (RL) offers a promising approach for such sequential decision-making problems. By interacting with an environment and receiving feedback, an RL agent develops a control policy that optimizes long-term performance. Elevator control naturally fits the RL framework: the controller must repeatedly decide movements while balancing competing objectives—minimizing waiting time and reducing unnecessary travel.

This project investigates tabular RL for elevator scheduling in a 10-floor building. The control task is formulated as an MDP. Q-Learning and SARSA are implemented and evaluated against FCFS, Nearest-Request, and Model Predictive Control (MPC). MPC is included as an optimization-based extension to benchmark RL against a planning approach requiring no training. The project emphasizes engineering rigor: multiple critical bugs producing nonsensical results were diagnosed and resolved, and these challenges are documented in detail.

II. Literature Review

A. Deep RL for Elevator Group Control

Vaartjes and Francois-Lavet [1] address Elevator Group Control Systems using Dueling Double Deep Q-Learning (DDQN), introducing "infra-steps" to bridge discrete decisions and continuous dynamics. Their work motivates the MDP formulation adopted here.

B. Deep Q-Learning for Optimization

Cao et al. [2] investigate DQN using a canonical three-peak traffic model. They emphasize comparing RL against hard-coded baselines—a strategy mirrored in our evaluation.

C. Tabular RL for Single-Elevator Control

Vasukiran et al. [3] compare model-based Q-value iteration with model-free Q-Learning for single-elevator systems, demonstrating that RL learns optimal policies without prior traffic knowledge.

III. Environment Formulation

A. MDP Formulation

The problem is formulated as $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the discrete action set, $\mathcal{P}$ is the stochastic transition function, $\mathcal{R}$ is the reward function, and $\gamma = 0.99$.

B. State Representation

The initial bitmask design encoded hall calls and car calls over all floors, growing beyond 200,000 states during training—too large for tabular convergence. The final representation uses compact binary signals:

$$s = \big(f,\ d,\ c_{\uparrow},\ c_{\downarrow},\ c_{\text{here}},\ \delta_{\uparrow},\ \delta_{\downarrow},\ \ell\big) \tag{1}$$

where $f \in \{0, \ldots, 9\}$ is the current floor; $d \in \{0, 1, 2\}$ encodes direction; $c_{\uparrow}, c_{\downarrow}, c_{\text{here}} \in \{0, 1\}$ indicate waiting passengers above, below, or at the floor; $\delta_{\uparrow}, \delta_{\downarrow} \in \{0, 1\}$ indicate in-

side passenger destinations; and $\ell \in \{0, 1, 2\}$ is a load bucket (empty/partial/full). Theoretical bound: $10 \times 3 \times 2^5 \times 3 = 2{,}880$ states; approximately 480–515 visited in practice.

C. Action Space

Three discrete actions: **0** move down, **1** stay/serve, **2** move up.

D. Reward Function

$$r_t = n_{\text{served}} \cdot r_{\text{svc}} - \mathbf{1}_{\text{moved}} \cdot r_{\text{mv}} - r_{\text{wt}} \cdot \min(N_{\text{wait}},\ 10) \quad (2)$$

where n_{served} is passengers delivered, $\mathbf{1}_{\text{moved}}$ indicates movement, and N_{wait} is total waiting passengers. Final values: $r_{\text{svc}} = 10.0$, $r_{\text{mv}} = 0.5$, $r_{\text{wt}} = 1.0$. Capping waiting penalty at 10 was essential to prevent reward collapse.

E. Transition Dynamics

Arrivals follow Poisson($\lambda = 0.3$) per step with random origin and destination. Episodes: 500 steps, capacity: 8 passengers.

IV. Methodology

A. Q-Learning (Off-Policy)

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big] \quad (3)$$

Using $\max_{a'}$ assumes optimal future behavior, producing aggressive policies that converge quickly but may overestimate values in stochastic environments.

B. SARSA (On-Policy)

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma Q(s',a') - Q(s,a)\big] \quad (4)$$

where $a' \sim \pi_\varepsilon(s')$. Accounting for exploration produces a conservative policy well-suited to stochastic environments.

C. Hyperparameters

Table 1: Training Hyperparameters

Parameter	Symbol	Value
Learning rate	α	0.1 (decaying)
Discount factor	γ	0.99
Initial epsilon	ε_0	1.0
Minimum epsilon	$\varepsilon_{\min}$	0.05
Epsilon decay	δ_ε	0.995
Training episodes	–	5,000
Steps per episode	–	500

Decaying learning rate for late-training stability:

$$\alpha_t = \max\big(0.01,\ \alpha_0 \cdot 0.9995^t\big) \quad (5)$$

D. Model Predictive Control

At each step MPC solves a receding-horizon problem over $H = 6$ steps:

$$a^* = \arg\max_{a_0 \in \mathcal{A}} \sum_{k=0}^{H-1} \gamma^k \hat{r}(\hat{s}_k, a_k) \quad (6)$$

using the known environment model (no arrivals assumed in horizon), with subsequent actions following a greedy nearest-request heuristic. Solved by exhaustive search over $|\mathcal{A}| = 3$ first actions. MPC requires no training but incurs per-step planning cost.

E. Baseline Controllers

FCFS serves the oldest pending call—simple and fair but spatially blind. **Nearest-Request** delivers inside passengers first, then moves to the nearest waiting floor.

V. Results and Analysis

A. Training Convergence

Both agents converge within 600 episodes (the high-ε exploration phase), with training reward stabilizing above zero—confirming service rewards consistently exceed penalties. SARSA maintains higher training reward throughout, consistent with conservative on-policy updates. The Q-table plateaued at 480–515 states, confirming full coverage within the budget.

B. Evaluation Results

Table 2: Controller Performance (50 Evaluation Episodes)

Controller	Served	Avg W	Max W	Moves
SARSA	**148.3**	**5.36**	**22.9**	483
Q-Learning	148.1	6.35	35.1	485
MPC	140.6	5.67	32.5	**374**
Nearest-Req.	145.0	7.86	59.4	480
FCFS	137.6	8.47	60.1	464

Avg W = avg waiting time (steps); Max W = max waiting time.

C. Per-Controller Analysis

SARSA achieves the best results across all primary metrics: lowest average wait (5.36), lowest maximum wait (22.9), and highest throughput (148.3). The on-policy update mechanism produces a consistent policy that avoids overshooting behavior.

Q-Learning nearly matches SARSA on throughput (148.1) but exhibits higher variance: average wait 6.35 and maximum wait 35.1 vs. SARSA's 22.9. Off-policy bootstrapping from $\max_{a'} Q(s',a')$ overestimates state values during exploration, producing occasional poor decisions in high-occupancy states.

MPC achieves the best movement efficiency—374 moves vs. 483–485 for RL agents, a 23% reduction—through 6-step planning. However, the horizon assumption (no future arrivals)

limits throughput to 140.6 served, below the RL agents who learned through experience to stay active throughout episodes.

RL vs. MPC comparison: RL agents serve 5% more passengers (148 vs. 141) but make 29% more movements. MPC is preferable when movement cost is the primary objective; SARSA is preferable for throughput and consistent service. This demonstrates that learned long-term behavior and optimal short-term planning are genuinely complementary.

D. Q-Value Heatmap Analysis

Heatmaps reveal rational spatial policies: DOWN action favored at floors 0–4, UP favored at floors 5–9, with boundary penalties correctly learned at floors 0 and 9. SARSA shows the same spatial structure as Q-Learning but with smaller magnitude differences between actions—consistent with conservative on-policy learning.

VI. Design Challenges and Insights

A. State Space Explosion

The bitmask representation grew beyond 200,000 states—far exceeding tabular convergence capacity. Binary signals reduced this to $\sim$500 states. *Key insight*: state space size is the single most impactful design factor for tabular RL.

B. Reward Shaping Failures

Two sequential bugs prevented learning: (1) without a waiting penalty, agents learned to barely move (6 movements/episode), serving only 2 passengers vs. FCFS's 137; (2) with an uncapped penalty, 160+ waiting passengers produced -160/step rewards that overwhelmed $+10$ service rewards, causing agents to stay still. Capping at 10 resolved both.

C. Python defaultdict Ghost Keys

The environment used `defaultdict(list)` for waiting passengers. Bracket read access (`waiting[f]`) automatically created empty lists at visited floors, making all controllers perceive phantom waiting passengers everywhere. Fixed by replacing all reads with `waiting.get(f)`.

D. Agent Evaluation Branch Bug

The trace function used `hasattr(agent, 'select_action')` to distinguish RL from baseline agents. Since both share this method, RL agents were incorrectly routed to the baseline branch—receiving the environment object as a state, producing all-zero Q-lookups, and defaulting to action 0 (DOWN) every step. The elevator sat at floor 0 for entire episodes. Fixed by `isinstance()` type checking.

VII. Conclusion

SmartLift successfully applies tabular RL to elevator scheduling. Starting from agents serving only 2 passengers per episode, principled fixes produced controllers outperforming all baselines on key metrics.

Key findings: (1) SARSA achieves best overall performance through conservative on-policy learning; (2) Q-Learning matches throughput but shows higher variance from off-policy overestimation; (3) MPC achieves superior movement efficiency but is bounded by its finite planning horizon; (4) state space engineering and reward shaping are the most impactful design factors; (5) subtle implementation bugs can produce deceptively convincing but incorrect results requiring systematic diagnosis.

Future work could explore multi-elevator coordination, DQN for continuous state formulations, and MPC with arrival rate priors to improve horizon modeling.

VII. References

[1] N. Vaartjes and V. Francois-Lavet, "Novel RL approach for efficient elevator group control systems," *arXiv preprint arXiv:2507.00011*, 2025.

[2] Z. Cao, R. Guo, C. M. Tuguinay, M. Pock, J. Gao, and Z. Wang, "Application of deep Q learning with simulation results for elevator optimization," in *Proc. SAI Intelligent Systems Conf.*, Springer, 2023, pp. 849–861.

[3] V. Vasukiran *et al.*, "Reinforcement learning for elevator control," in *2023 Int. Conf. Computational Intelligence for Information, Security and Communication Applications (CIISCA)*, IEEE, 2023, pp. 379–382.